PROJECT REPORT: EMBEDDED SYSTEMS & IOT DEVICE DESIGN

Submitted By: Junior Embedded Engineer

Organization: Maincrafts Technology

Task Reference: Embedded Systems Task-1

1. Introduction & Theoretical Foundation

What is an Embedded System?

An embedded system is a specialized, computer-based system consisting of hardware integrated with dedicated software (firmware) designed to execute a specific, pre-defined function. Unlike general-purpose computers (e.g., personal laptops) designed to run an infinite variety of applications, an embedded system is highly optimized for efficiency, reliability, and deterministic real-time operations.

Real-World Applications

Embedded systems serve as the invisible backbone of modern infrastructure. Key industries include:

- **Smart Home Automation:** Smart thermostats, automated security panels, and automated environmental controllers.
- **Automotive Electronics:** Electronic Control Units (ECUs), Anti-lock Braking Systems (ABS), and advanced driver-assistance systems (ADAS).
- **Healthcare Systems:** Critical medical electronics such as implantable pacemakers, automated insulin pumps, and real-time patient vital monitors.
- **Robotics & Industrial Automation:** Programmable Logic Controllers (PLCs), robotic assembly arms, and automated guided warehouse vehicles.

Microcontroller (MCU) vs. Microprocessor (MPU)

Selecting the appropriate processing architecture is a fundamental step in embedded system design. The table below highlights their key functional distinctions:

Technical Feature	Microcontroller (MCU)	Microprocessor (MPU)
Architectural Design	An all-in-one chip integrating the CPU core, RAM, ROM, and I/O peripherals onto a single silicon substrate.	A standalone CPU core chip that requires external ICs for RAM, ROM, storage, and peripheral interfaces.
Production Cost	Significantly lower cost per unit, highly optimized for dedicated tasks.	Higher implementation cost due to the necessity of supporting external components.
Power Consumption	Extremely low; often designed with deep-sleep modes to run on battery power for years.	High power requirements; typically demands stable external power grids and active thermal cooling.
Core Examples	ATmega328P (Arduino UNO), ESP32, RP2040.	Intel Core series, AMD Ryzen, ARM Cortex-A series (found in smartphones).

2. Component Study

Microcontrollers Evaluated

- **Arduino UNO:** An entry-level 8-bit development platform utilizing the ATmega328P microcontroller. It features an open-source toolchain and a massive library ecosystem, making it the industry-standard baseline for low-complexity hardware prototyping.
- **ESP32:** A high-performance 32-bit dual-core microcontroller featuring native Wi-Fi and Bluetooth connectivity. This platform is the primary choice for modern Internet of Things (IoT) field deployments that demand cloud communication capability.

- **Raspberry Pi Pico:** A low-cost, high-efficiency board powered by the custom dual-core RP2040 chip. It offers flexible digital interfacing through unique Programmable I/O (PIO) blocks, ideal for high-speed deterministic execution.

Sensors Evaluated

- **Temperature Sensors (e.g., TMP36 / DHT11):** Transducers that capture environmental thermal energy and convert it into a proportional analog voltage or digital signal for computational tracking.
- **Motion Sensors (e.g., PIR - Passive Infrared):** Electronic modules that detect ambient infrared energy signatures emitted by moving human bodies or warm objects within their field of view.
- **Light Sensors (e.g., LDR - Light Dependent Resistor):** Photo-resistors that exhibit a change in electrical resistance correlated directly to the intensity of ambient light falling on their surfaces.

3. Mini Project Design: Smart Motion Detection Alarm System

System Selection

- **Project Chosen:** Motion Detection Alarm System
- **Development Platform:** Tinkercad Circuits (Digital Simulator Environment)

Working Principle

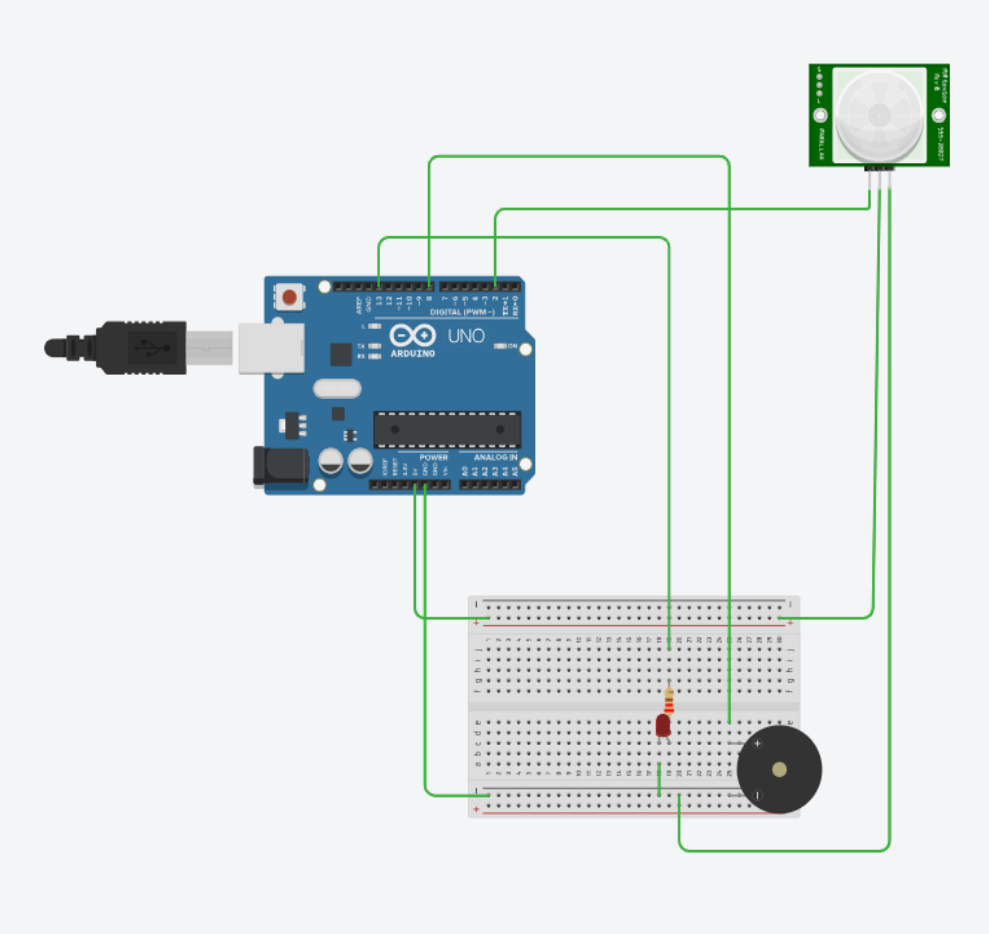
The system is engineered as an automated perimeter monitoring node.

1. The **PIR Sensor** acts as the input node, monitoring a specific geographic zone for fluctuations in ambient infrared thermal patterns.
2. When an object or person steps into the detection cone, the sensor registers an IR differential and shifts its physical output signal from LOW to HIGH.
3. The **Arduino UNO** reads this logic shift via its digital input pin array.
4. Upon evaluating the HIGH conditional state, the MCU executes its code to immediately turn on two separate output peripherals simultaneously: a high-brightness warning **LED** and an audible **Piezo Buzzer** to notify personnel of an intrusion.

Circuit Wiring Specification Table

The virtual prototype was arranged on a solderless breadboard using the following pin map configurations:

Source Component	Component Pin	Destination Component	Destination Pin / Rail	Wire Function
Arduino UNO	5V	Breadboard	Positive (+) Rail	Primary VCC Bus
Arduino UNO	GND	Breadboard	Negative (-) Rail	System Ground
PIR Sensor	VCC / Power	Breadboard	Positive (+) Rail	Sensor Power
PIR Sensor	GND / Ground	Breadboard	Negative (-) Rail	Sensor Ground Reference
PIR Sensor	OUT / Signal	Arduino UNO	Digital Pin 2	Digital Input Signal
Resistor (220 Ω)	Terminal 1	Arduino UNO	Digital Pin 13	Output Control Path
Resistor (220 Ω)	Terminal 2	LED	Anode (+)	Current-Limited Power
LED	Cathode (-)	Breadboard	Negative (-) Rail	LED Ground Return
Piezo Buzzer	Positive (+)	Arduino UNO	Digital Pin 8	Audio Output Control
Piezo Buzzer	Negative (-)	Breadboard	Negative (-) Rail	Buzzer Ground Return



Tinkercad simulation circuit schematic for the Motion Detection Alarm System.

4. Source Code Implementation

The firmware is written in standard Arduino C++ and uploaded directly into the integrated development space of the simulator

/* * Maincrafts Technology - Embedded Systems & IoT Task 1

* Project: Motion Detection Alarm System (Simulation)

* File: main.ino

* Author: Junior Embedded Engineer

*/

```
// Global Constant Declarations for Pin Allocations
const int pirInputPin = 2;   // Signal interface pin for the PIR Sensor
const int ledAlertPin = 13;  // Drive control pin for the warning LED
const int buzzerAlertPin = 8;// Drive control pin for the warning Buzzer
// State Variable to hold real-time sensor evaluations
int motionState = 0;
void setup() {
  // Establish pin modes based on I/O flow mapping
  pinMode(ledAlertPin, OUTPUT);       // Configure pin 13 to drive LED current
  pinMode(buzzerAlertPin, OUTPUT);    // Configure pin 8 to drive Audio Piezo
  pinMode(pirInputPin, INPUT);        // Configure pin 2 to sample PIR voltage levels

  // Set up data transmission link back to host terminal for operational logging
  Serial.begin(9600);
  Serial.println("System Security Monitor Online. Calibrating peripherals...");
}
void loop() {
  // Continually evaluate the digital state of the physical PIR pin
  motionState = digitalRead(pirInputPin);
  // Conditional Logic Branch: Act if sensor registers structural movement
  if (motionState == HIGH)
 {
    // Action 1: Drive LED high to ignite the light source
    digitalWrite(ledAlertPin, HIGH);
    // Action 2: Drive Piezo element high to produce sound waves
    digitalWrite(buzzerAlertPin, HIGH);
    // Action 3: Log timestamped diagnostic telemetry string to the console
    Serial.println("[ALERT] Threat Detected inside Monitored Sector!");
```

```
  // Safety Delay: Prevents short stutter loops and maintains crisp alert states
  delay(1000);
 }
 else
 {
  // Return to Default Safety State: Shutdown active alert pins
  digitalWrite(ledAlertPin, LOW);
  digitalWrite(buzzerAlertPin, LOW);
 }
}
```

5. Industry Applications & Scalability

The simulated perimeter logic is highly scalable and directly matches several industrial market solutions:

1. **Commercial Security Frameworks:** Can be scaled up into multi-node hardware arrays to secure facilities, linking back via wireless protocols (like ESP32 Wi-Fi) to trigger central monitoring stations.
2. **Smart Lighting Infrastructure:** The logic can be inverted to act as an energy-conservation tool in corporate offices, automatically switching overhead lighting relays off during periods of zero motion to minimize energy footprints.
3. **Industrial Machine Safeguards:** Can be positioned near active high-voltage automated manufacturing lines to instantly isolate or shut down heavy equipment if human operators accidentally step into hazardous operational zones